

Universidad Tecnológica de Bolívar

Parcial ingeniería de software

Profesor: Aisner Jose Marrugo Juliao

Integrantes:

Josh Robinson Ortega Castellón

Luis Eduardo Mendoza Angulo

Fundamentos

Problemas, actores y alcance

Un colectivo de escritores prepara una ficción interactiva para un festival universitario y no sabe programar. Hoy, cambiar una escena o una decisión exigiría modificar el programa. Además, un error en el contenido podría cerrar la aplicación frente al lector. Se necesita un motor que lea narraciones desde archivos, muestre solo las opciones compatibles con lo ocurrido, registre las consecuencias de cada elección y avise al escritor qué corregir cuando un archivo es inválido.

Actores

Actor	**Rol**	**Interés principal**
Escritora principal	Edita los archivos de historia	Cambiar escenas y decisiones sin tocar código
Diseñador de la experiencia	Define condiciones y consecuencias	Opciones que aparecen según lo descubierto, o sea decisiones con peso
Coordinadora del festival	Responsable de la puesta en marcha	Errores diagnosticables, por ejemplo, que el programa no cierre
Lector	Juega la historia	Leer, elegir, llegar a un final y reiniciar

Alcance

Dentro (v1)	Fuera (declarado por el cliente)
Carga de historias desde archivo	Cuentas, guardado entre ejecuciones, base de datos
Condiciones de igualdad y efectos sobre variables booleanas	Lenguaje de expresiones, editor visual, audio, imagenes
Opciones filtradas por estado, finales y reinicio	Conexión a internet
Validación de contenido con mensajes de causo y campo	Análisis automatico de todos los caminos posibles
Historia de referencia (escenas) y segunda historia (≥ 3 escenas)	Retroceso de decisiones (conflicto abierto, sin implementar)

Dos riesgos con mitigación

ID	**Riesgo**	**Prob.**	**Impacto**	**Mitigación**	**Como sabremos que ocurre**
R1	**Acoplamiento del motor a la historia de**	Media	Alto	Contenido, reglas e interfaz en módulos	Un identificador

	referencia. Si se programan reglas específicas para E1, E3, etc., la segunda historia y el cambio de la hora 6 rompen el motor.			separados. Condiciones y efectos evaluados solo desde los datos. Probar desde temprano con la segunda historia.	de escena aparece en el código fuente del motor.
R2	**Cambio tardío con tiempo limitado.** La solicitud llega en la hora 6, con la v1 posiblemente incompleta, y puede afectar requisitos, modelos y pruebas a la vez.	Alta	Alto	Guardar una versión inicial en la hora 6 con etiqueta en Git. Mantener trazabilidad desde el comienzo. Reservar 1 h 30 min para cambio y pruebas. Dejar IDs de RF libres (RF20+) para el cambio.	Backlog sin tarea de cambio al llegar a la hora 7.

Priorización de atributos de calidad

Prioridad	Atributo	Justificación (voz del cliente)	Se medirá con
1	Modificabilidad del contenido	La escritora quiere agregar escenas sin pedir cambios al programa	Una historia nueva funciona sin edtiar el motor
2	Confiabilidad/robustez ante errores	La coordinadora no acepta cierres inesperados	Entradas inválidas o archivos defectuosos no terminan el programa
3	Claridad de los errores (usabilidad para el escritor)	Debe saber qué corregir	El mensaje indica causa y campo o ID
4	Testeabilidad	Es exigida por la entrega e indispensable para el cambio	Cobertura de los escenarios U1-U5, I1 y el nuevo
Baja	Rendimiento, seguridad portabilidad	Historias pequeñas, uso local, sin red ni cuentas	No se mide en esta versión

Decisión de responsabilidad profesional

Autoría: usamos un asistente de IA (Claude) para borradores de documentos, código y diagramas. Cada uso se registra en ia.md con su finalidad, lo que se aceptó o cambió y cómo se verificó, y no se incorpora código que no entendamos ni hayamos probado. Se descartó usarlo sin registro porque haría pasar como propio material generado. **Datos:** el motor no pide ni guarda información del lector (sin cuentas ni guardado entre ejecuciones), así que no maneja datos personales.

Requerimientos

ID	Necesidad	Requisito	Criterio de aceptación
RF01	N01	Cargar una historia desde un archivo indicado, sin modificar código para cambiar de historia.	Dados la historia de referencia y la segunda historia, ambas se cargan y muestran su título.
RF02	N01	Iniciar la partida en inicio con las variables iniciales.	La partida comienza en E1 con ambas variables en falso.
RF03	N02	Mostrar el texto de la escena actual.	En E1 se muestra el texto de “La señal”.
RF04	N02	Mostrar solo las opciones cuyas condiciones se cumplen todas; sin condiciones, siempre se muestra.	En E3 con conoce_origen = verdadero se ve “Activar con código” y no “Activar sin verificar” (CA01).
RF05	N03	Aplicar los efectos de la opción elegida y llevar a su destino.	En E3 con origen conocido, “Activar con código” deja confia_en_la_voz = verdadero y va a E4.
RF06	N03	Rechazar una opción inexistente o no habilitada, aunque se envíe su ID al motor, sin alterar el estado.	En E3 sin origen, enviar “Activar con código” se rechaza y sigue en E3 (CA02).
RF07	N03	Tratar una entrada de lector no válida como error recuperable, sin alterar la partida.	Ingresar "abc" o "7" muestra un mensaje y vuelve a pedir la elección.
RF08	N04	Conservar las variables entre escenas y evaluar condiciones con el estado acumulado.	E1→E2 (investigar)→E3 muestra “Activar con código”; con “continuar” muestra “Activar sin verificar”.
RF09	N05	Mostrar el texto de una escena final y avisar que la partida terminó, sin opciones.	En E4, E5 o E6 se muestra el texto y el aviso de fin.
RF10	N05	Permitir llegar a los tres finales según las decisiones.	Las tres rutas desde E3 alcanzan E4, E5 y E6.
RF11	N05	Informar el bloqueo cuando una escena no final no tenga opciones habilitadas, sin cerrarse.	Una escena de prueba con opciones imposibles muestra el aviso de bloqueo.
RF12	N05	Permitir reiniciar tras un final o bloqueo, restaurando escena y variables iniciales.	Tras terminar en E4, reiniciar deja E1 y variables en falso.
RF13	N06	Rechazar antes de jugar un archivo con alguno de los 9 defectos de la tabla inferior.	Cada archivo defectuoso es rechazado.
RF14	N06	Indicar en cada rechazo la causa y el campo o ID involucrado.	Destino E99 en la opción investigar de E2 aparece en el mensaje.

RF15	N06	No crear partida con un archivo rechazado; terminar la carga de forma controlada.	Con destino E99 no hay partida ni cierre inesperado (CA03).

Requerimientos no funcionales

ID	Enunciado	Métrica y umbral	Procedimiento y evidencia
RNF01 Modificabilidad	Cambiar de historia o editar escenas, opciones, condiciones y efectos modificando solo el archivo de contenido.	0 archivos de código modificados; 0 IDs E1 a E6 en el código del motor.	git diff --stat vacío tras cargar la segunda historia; prueba test_motor_sin_ids_de_historia. Evidencia: evidencias/rnf01.txt.
RNF02 Confiabilidad	Ante archivos defectuosos o entradas inválidas, terminar de forma controlada, sin *traceback* ni cierre inesperado.	20 de 20 casos (11 archivos defectuosos que cubren las 9 categorías, 7 entradas inválidas del lector y 2 IDs inválidos enviados al motor).	Pruebas test_rnf02_archivo_defectuoso_termina_controlado (ejecuta el programa por subproceso y exige que no aparezca Traceback), test_rnf02_entrada_de_lector_invalida_no_altera_la_partida y test_rnf02_id_invalido_en_el_motor_es_error_controlado. Evidencia: evidencias/pytest_ultima_ejecucion.txt.
RNF03 Claridad de errores	Todo mensaje de rechazo incluye causa y campo o ID, en español y sin nombres internos	9 de 9 mensajes con causa e ID; 0 con nombres de clases o excepciones.	Prueba test_rnf03_mensajes_claros y revisión cruzada entre integrantes. Evidencia: evidencias/rnf03_revision.md.

	del código.		

Preguntas y supuestos

#	Tipo	Contenido
1	Supuesto	Se informan todos los errores del archivo, no solo el primero.
2	Pregunta al cliente	¿El retroceso debe estar siempre disponible o solo en ciertas decisiones? Acuerdo provisional: sin retroceso en la v1.
3	Supuesto	Las opciones se muestran en el orden del archivo.
4	Supuesto	El lector elige por número; la interfaz lo traduce al ID de la opción.

Defecto detectado en la revisión cruzada

Defecto: La palabra aparecer es ambigua al describir las opciones disponibles.

Corrección: Se reemplazó por “se muestra”, y RF04 define que solo se muestran las opciones cuyas condiciones se cumplen todas

Procesos

Proceso elegido y alternativa comparada

Proceso elegido: desarrollo incremental con tablero Kanban. Se construye la solución en incrementos pequeños y verificables (carga, motor, interfaz, validación, pruebas). Cada incremento pasa por un tablero con límite de trabajo en curso. El cambio de la hora 6 entra como una tarea más, sin romper el flujo.

Alternativa comparada: Scrum. Se trabaja en sprints de duración fija, con un *sprint backlog* comprometido al inicio, y con eventos como planificación, reunión diaria,

Criterio	Incremental+Kanban(elegido)	Scrum (descartada)
Cambio en la hora 6	Entra como tarjeta nueva en cualquier momento	El sprint en curso se protege de cambios; habría que replanificar o cancelar el sprint
Duración y cadencia	Flujo continuo, sin iteraciones fijas	Requiere sprints cortos (ej.: 3 h), con planificación y revisión en cada uno
Carga de ceremonias	Mínima: tablero y límite de trabajo en curso	Eventos y roles (Scrum Master, Product Owner) que no se justifican en una pareja y 12 h
Control del trabajo	Límite de 2 tareas por persona	Capacidad fijada por sprint

Por qué se elige Kanban y no Scrum: el parcial dura 12 horas, lo realiza una pareja y contiene un cambio programado. Scrum añadiría ceremonias y roles que consumirían tiempo, y su compromiso de sprint choca con una solicitud que llega a mitad del trabajo. Kanban permite absorber el cambio como una tarea más y mantener siempre una versión ejecutable.

Como se realiza cada actividad del proceso

Actividad	Como se hará en esta entrega	Evidencia
Especificación	Se redactan RF y RNF desde las necesidades N01 a N06, con criterios de aceptación. Se actualizan al llegar el cambio (RF20+).	Informe, Unidad 4; matriz de trazabilidad
Desarrollo	Incrementos en orden: modelo y carga, motor de reglas, interfaz de consola, validación. Arquitectura de monolito modular (ver 2.3).	Repositorio Git con commits por incremento
Validación	Pruebas automatizadas (U1 a U5, I1), verificación de los tres RNF y recorrido manual de los tres finales.	`evidencias/pytest_ultima_ejecucion.txt`, video
Evolución	En la hora 6: etiqueta Git de la versión inicial, análisis de impacto y cambio. Se actualizan requisitos, modelos, pruebas y trazabilidad.	Etiqueta `v1-hora6`, sección de gestión del cambio

Ajuste de arquitectura

Pasamos a un **monolito modular** en lugar de capas con despliegue separado. Es un solo programa Python con módulos de responsabilidad clara, sin servicios aparte ni infraestructura. Se justifica porque el proyecto es pequeño y el tiempo es corto. Los módulos siguen aislando contenido, reglas e interfaz (necesario para RNF01 y el riesgo R1). La decisión se documenta con alternativas y consecuencias en la Unidad 6.

Agilidad

Backlog y tablero: tablero Kanban en GitHub Projects con columnas Backlog, Ready, In progress, In review y Done; cada tarjeta tiene un responsable. El cambio de la hora 6 (RF20) se trabajó dentro de las tarjetas de implementación (#4) y de pruebas (#5). Al cierre, las 9 tarjetas quedaron en Done. Capturas inicial, intermedia y final en el Anexo A3.

Práctica aplicada	Cómo la aplicamos	Por qué
Tablero Kanban con límite de trabajo en curso	Máximo 2 tarjetas en curso por persona; una tarjeta pasa a Done solo si cumple la definición de terminado.	Hace visible el avance sin reuniones y absorbió el cambio de la hora 6 como una tarjeta más.
Pruebas automatizadas en cada incremento	Carga, motor, consola, validación y deshacer llegaron con sus pruebas; la suite se ejecuta antes de cada commit.	Permitió incorporar RF20 comprobando que nada anterior se rompía (74 pruebas).

Alternativa descartada: estimación con planning poker. Con 12 horas y dos personas la sesión de estimación consume tiempo sin cambiar decisiones, y el flujo continuo no necesita fijar capacidad; por eso el campo Estimate del tablero queda en 0.

Definición de terminado: una tarjeta pasa a Done solo si (1) el código está en main y se ejecuta con el comando del README; (2) tiene pruebas automatizadas y la suite completa pasa; (3) sus requisitos tienen criterio de aceptación y aparecen en la matriz de trazabilidad; (4) los modelos y la documentación afectados están actualizados; (5) el otro integrante lo revisó.

Aprendizaje y acción de mejora: algunas tarjetas pasaron a Done sin cumplir la definición (modelado sin diagramas, segunda historia con el README vacío), y el tablero mostraba más avance del real. Acción: antes de mover una tarjeta a Done, el otro integrante revisa la definición de terminado y anota en la tarjeta la ruta del archivo o la prueba que la demuestra.

Modelado

Los modelos están en el Anexo A1 y usan los nombres de clases, métodos y estados del código entregado, incluido el cambio de la hora 6. **D1 Contexto**: actores, archivo JSON y límites del sistema (RF01, RF13–RF15). **D2 Casos de uso**: cargar historia (incluye validar), elegir opción (extendido por ver fin o bloqueo), deshacer y reiniciar (RF01–RF15, RF20). **D3 Clases**: Historia, Escena y Opcion inmutables y Sesion con el retroceso (RF02, RF04–RF08, RF20). **D4 Secuencia de elegir una opción**: consola, Sesion y reglas, con la instantánea para deshacer (RF04–RF08, RF20). **D5 Estados de una sesión**: En juego, Final y Bloqueada con elegir, deshacer y reiniciar (RF09–RF12, RF20).

Arquitectura

Monolito modular de cinco paquetes (diagrama de componentes D6, Anexo A1): dominio (Historia, Escena, Opcion y errores propios), contenido (lee y valida el JSON), motor (reglas y Sesion), interfaz (consola) y __main__ (une las piezas). motor no conoce contenido ni interfaz, y ningún módulo contiene IDs de una historia concreta.

	DA1: monolito modular	**DA2: validar todo antes de jugar**
Contexto	Proyecto pequeño, 12 horas, una pareja, ejecución local sin red.	La escritora no programa y la coordinadora no acepta cierres inesperados (N06, RNF02, RNF03).
Alternativas	Capas con despliegue separado (API + cliente); un único script sin módulos.	Detectar errores durante la partida; detenerse en el primer error.
Elección	Un programa Python con paquetes separados por responsabilidad.	validador.py reúne todos los problemas y no se crea la Sesion si hay alguno.
Consecuencias	Sin infraestructura que mantener; cambiar de interfaz solo toca interfaz. Los límites dependen de disciplina, por eso los vigila test_motor_sin_ids_de_historia.	El motor supone una historia válida y queda simple; la escritora corrige todo en una pasada. Un error de sintaxis JSON se informa de uno en uno.

Separación demostrada: la segunda historia (historias/historia2.json, 4 escenas, una condición y un efecto, sin IDs E1–E6) se juega con el mismo motor sin modificar código: evidencias/rnf01.txt y test_i1_segunda_historia_funciona_sin_tocar_el_motor. El cambio de la hora 6 solo tocó motor e interfaz.

Implementación y pruebas

Decisión de diseño detallado: las reglas (esta_habilitada, aplicar_efectos) son funciones puras y Sesion es la única clase con estado. La consola recibe leer y escribir como parámetros y solo usa la interfaz pública de Sesion, así se prueba con entradas simuladas y cambiar la interfaz no toca el motor. En la revisión se quitó un acceso de la consola al atributo privado sesion._historia.

Bibliotecas y licencias: el programa usa solo la biblioteca estándar de Python 3.14 (licencia PSF). Las pruebas usan pytest 9.1.1 (MIT) y sus dependencias pluggy e iniconfig (MIT), packaging (Apache-2.0 o BSD-2), Pygments (BSD-2) y colorama (BSD-3). Instalación, ejecución y pruebas: README.md.

Escenario	**Archivo de prueba (tests/)**	**Resultado**
U1 condiciones	test_reglas.py	Aprobada
U2 elección válida, U3 opción inválida	test_sesion.py (rutas a E4, E5, E6; CA02)	Aprobada

U4 errores de contenido	test_validacion.py (11 archivos, 9 categorías)	Aprobada
U5 reinicio, U6 deshacer, bloqueo	test_sesion.py	Aprobada
I1 integración	test_integracion.py (archivo real, tres finales, deshacer, E99)	Aprobada
RNF01 a RNF03	test_rnf.py	Aprobada
Total	evidencias/pytest_ultima_ejecucion.txt	74 de 74

Verificación de los RNF: RNF01: 0 archivos de código modificados y 0 IDs E1–E6 en src/ (evidencias/rnf01.txt). RNF02: 20 de 20 casos terminan de forma controlada, sin Traceback. RNF03: 11 de 11 mensajes con causa y campo o ID y sin nombres internos; revisión cruzada en evidencias/rnf03_revision.md. Los tres finales se recorrieron a mano con las rutas del README (1-1-1 → E4, 1-2-1 → E5, 1-1-2 → E6). No hay fallos pendientes; las limitaciones están en el README.

Después del Cambio

La solicitud de cambio **reemplaza el acuerdo provisional** de no implementar retroceso (Unidad 4): el lector debe poder deshacer su última decisión con todas sus consecuencias.

ID	Necesidad	Requisito	Criterio de aceptación
RF20	N03,N04	El sistema debe permitir deshacer la última elección válida, restaurando exactamente la escena y las variables anteriores, en un solo nivel y también desde un final.	**Dado** E2, **cuando** se elige «Investigar archivo» (E3, conoce_origen verdadero) y se deshace, **entonces** la sesión vuelve a E2 con conoce_origen falso. Un segundo deshacer no cambia nada. Al elegir «Continuar sin investigar», E3 muestra «Activar sin verificar» y «Cortar comunicación».

Reglas que cubre RF20

Regla de la solicitud	**Comportamiento verificado**
Restauración	Se recupera la escena y todas las variables previas a la última elección válida.
Un solo nivel	Tras deshacer no hay otro retroceso hasta una nueva elección válida. No hay rehacer.
Inicio y reinicio	Al comenzar no hay retroceso. Reiniciar elimina el historial y restaura el estado inicial.
Entrada invalida	Una opción inexistente o no habilitada no modifica la sesión ni sustituye el historial.

Desde un final	Desde E4, E5 o E6 se regresa a E3 con los valores previos.

Análisis de impacto

Artefacto	¿Afectado?	Como
Requisitos	**sí**	Se agrega RF20 y se amplía RF12. Se reemplaza el supuesto del retroceso.
Modelos	**sí**	Casos de uso (D2): nuevo caso Deshacer decisión. Clases (D3): Sesion guarda el estado anterior (_anterior, puede_deshacer, deshacer()). Secuencia (D4): se guarda una copia antes de aplicar la opción. Estados (D5): transiciones deshacer con la guarda puede_deshacer.
Componentes	**sí**	motor/sesion.py (método deshacer, copia del estado en elegir, limpieza en reiniciar) e interfaz/consola.py (comando d).
Sin cambios	**no**	dominio/, contenido/, motor/reglas.py e historias/*.json. El formato de contenido no cambia.
Pruebas	**sí**	U6 nueva, U5 e I1 extendidas.
Documentacion	**sí**	docs/formato_contenido.md (regla del retroceso), README (controles y limitaciones).

Decisión de diseño: la sesión guarda una copia de `(escena, variables)` justo antes de cada elección válida. Deshacer la restaura y la descarta, lo que garantiza un solo nivel. La copia se guarda solo después de validar la opción, así una entrada inválida no toca el historial. Se descartaron guardar una pila completa (la solicitud exige un solo nivel) y deshacer volviendo a calcular efectos al revés (frágil con efectos que sobrescriben valores).

Versiones y trazabilidad: la versión disponible al recibir la solicitud se conserva con la etiqueta v1-hora6 (commit 6eb8b14): informe v1, historia de referencia y estructura de módulos, todavía sin código ejecutable. La versión posterior es la rama main desde el commit ee4cf43, que ya incorpora RF20. La matriz de trazabilidad de RF01–RF15, RF20 y RNF01–RNF03, con modelos, archivos y pruebas, está en el Anexo A2.

Aportes y fuentes

Integrante	Aportes (evidencia en evidencias/git_historial.txt)
Josh Robinson Ortega Castellón	Versión v1-hora6 (6eb8b14); carga, validación, motor, consola, deshacer y segunda historia (ee4cf43); configuración de pytest, archivos inválidos, guía para la escritora y pruebas (d63c602, 647eaed, 31c0e92); historial de Git (6e031d5).
Luis Eduardo Mendoza Angulo	Estructura inicial del repositorio (482aade); README, evidencias de RNF, prueba de bloqueo, análisis del cambio y trazabilidad (c2d46a0); diagramas D1–D6 (07ff7fc); limpieza del repositorio (5560580).

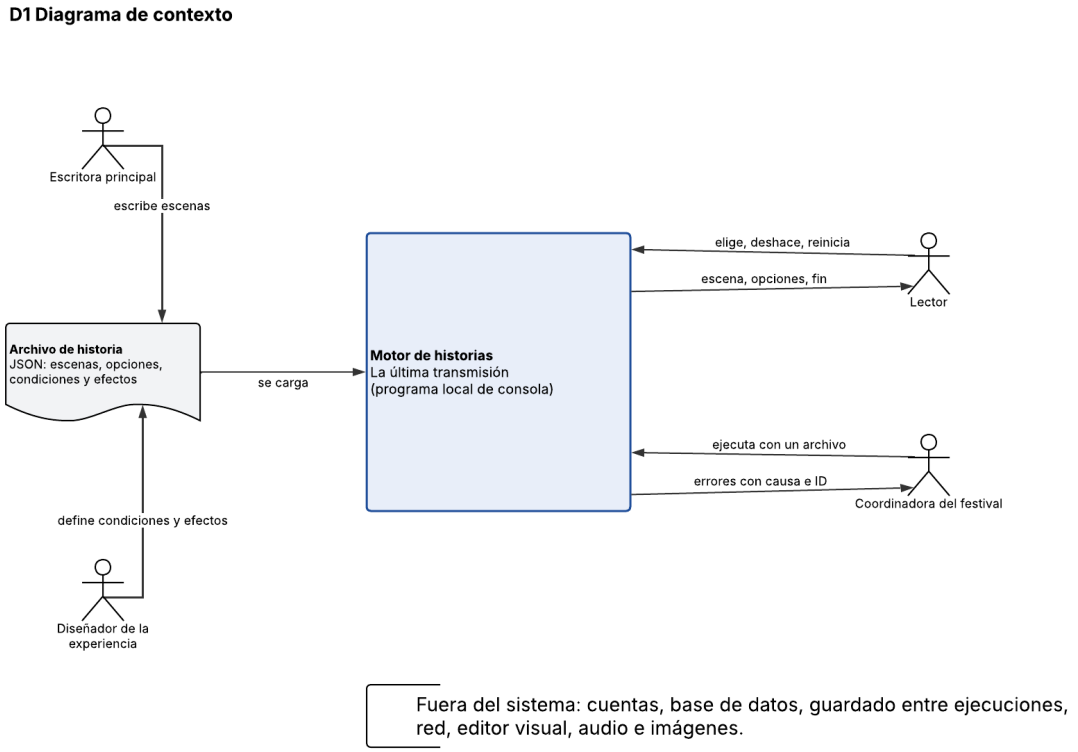
Ambos	Redacción del informe, revisión cruzada de requisitos (Unidad 4) y tablero.

Fuentes y herramientas: Python 3.14 y pytest; Git y GitHub (código y tablero); Lucidchart (diagramas); Claude como asistente de IA, con cada uso, su finalidad y su verificación registrados en ia.md. La historia de referencia es material del docente; la segunda historia es propia.

Anexos

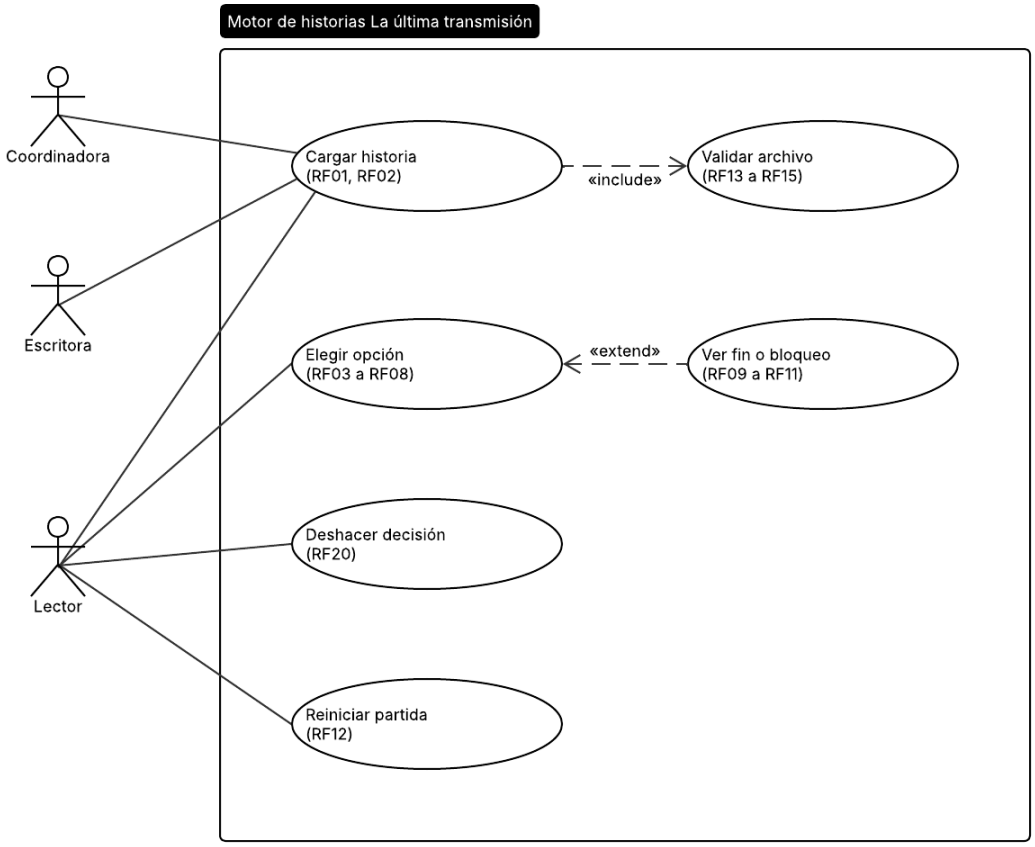
Repositorio	https://github.com/luis20072002/last_transmition
Tableros	https://github.com/luis20072002/last_transmition/tree/main/docs/tablero
Diagramas	https://github.com/luis20072002/last_transmition/tree/main/docs/anexo/diagramas
Evidencias	https://github.com/luis20072002/last_transmition/tree/main/evidencias
Video de demostración	https://youtu.be/0zrbxGzc0fI

A1. Diagramas



D1. Diagrama de contexto

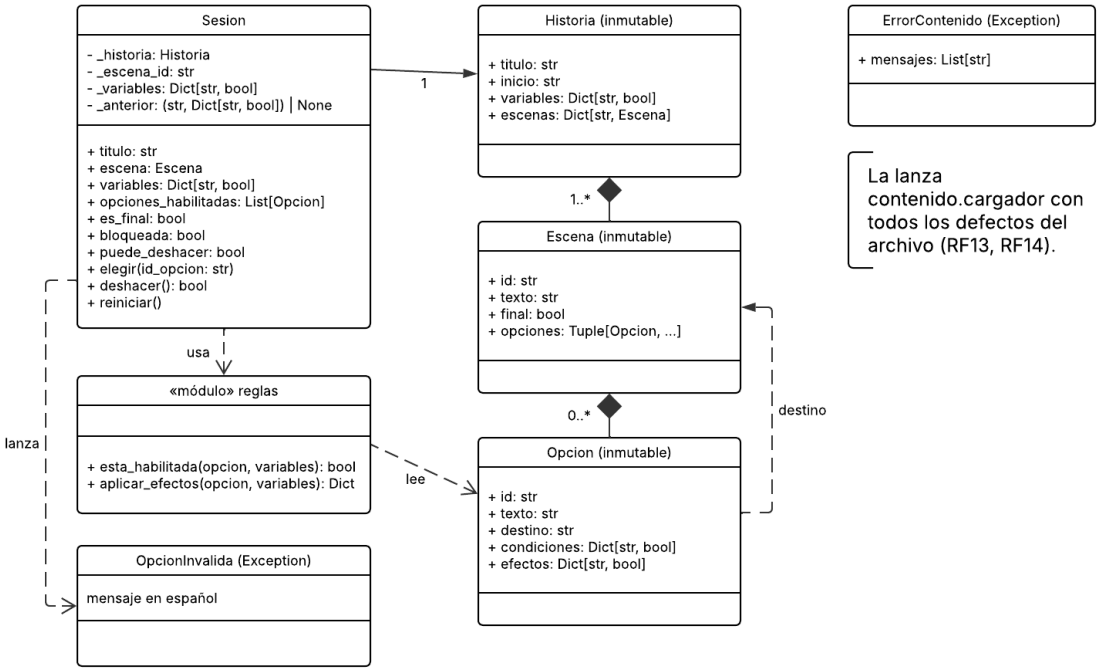
D2 Casos de uso



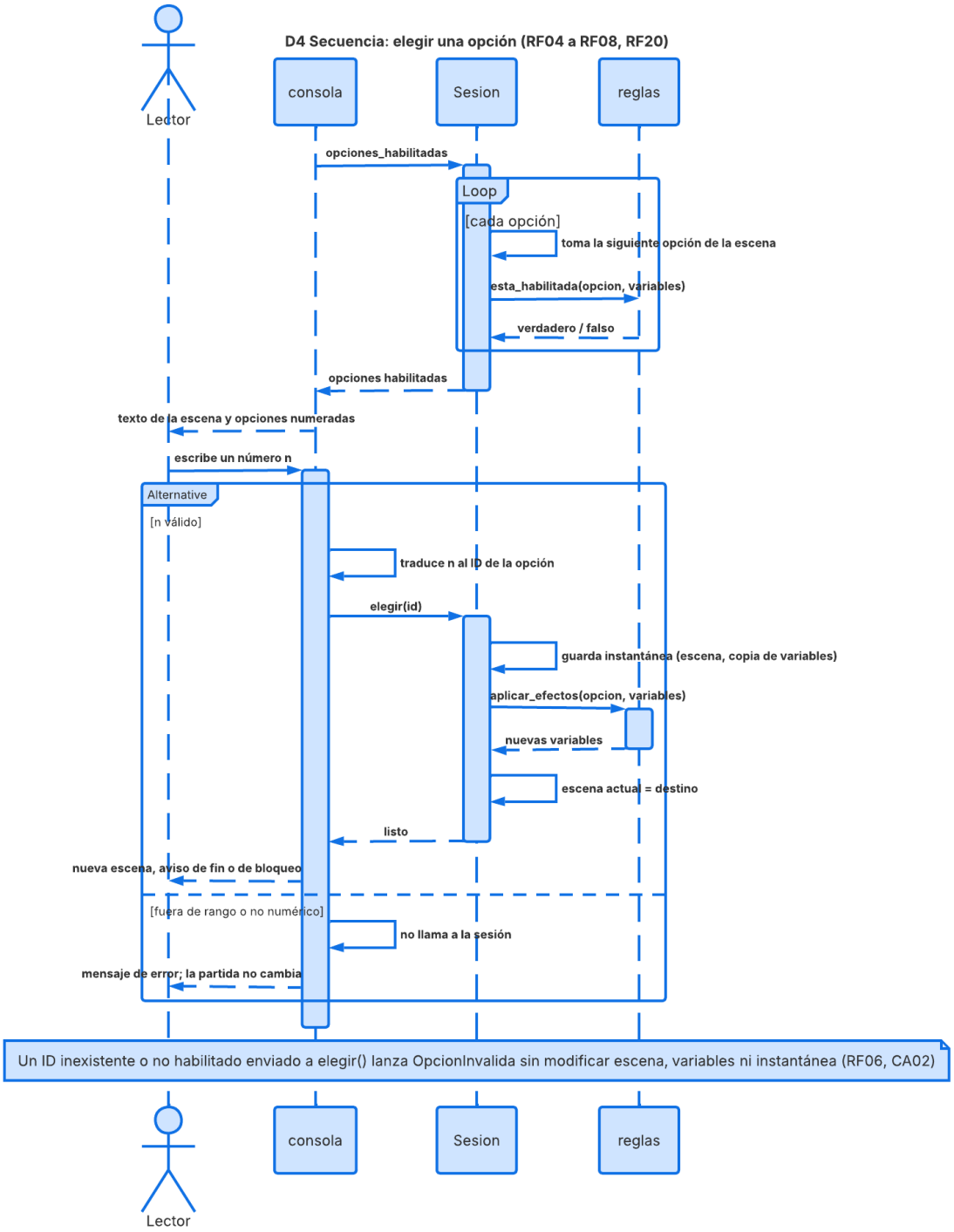
El diseñador edita el mismo archivo que la escritora. El archivo se prepara fuera del sistema con un editor de texto.

D2. Casos de uso

D3 Clases del dominio

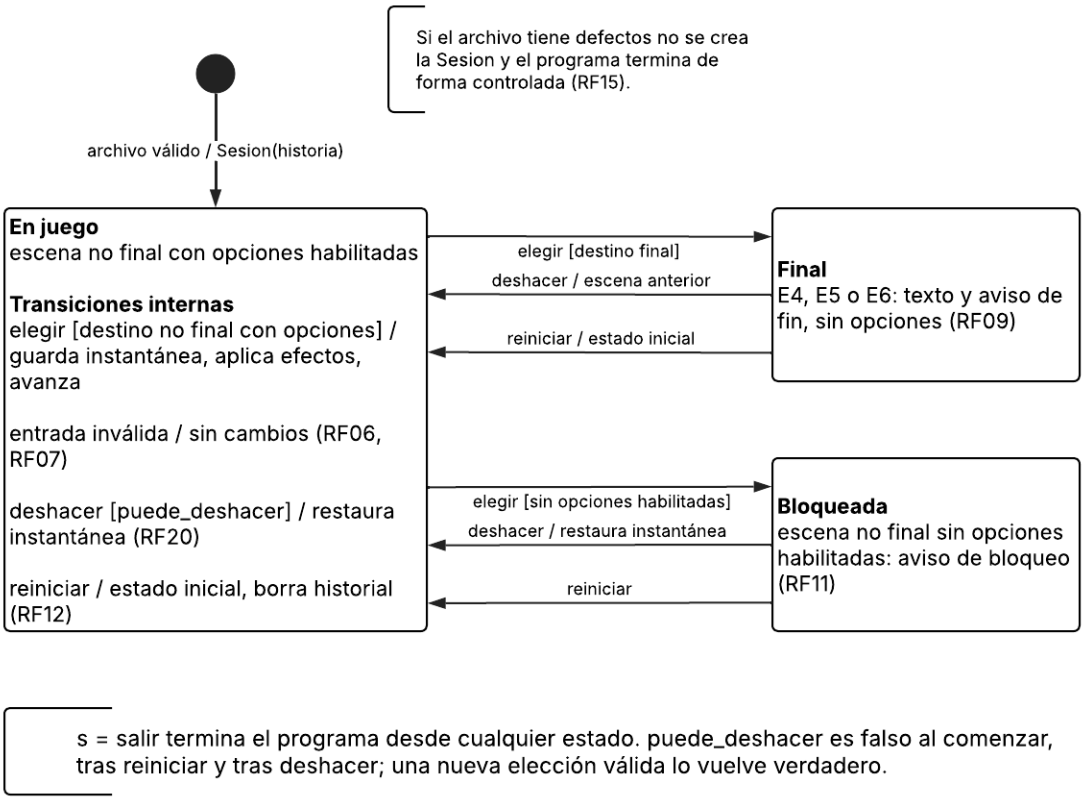


D3. Clases del dominio

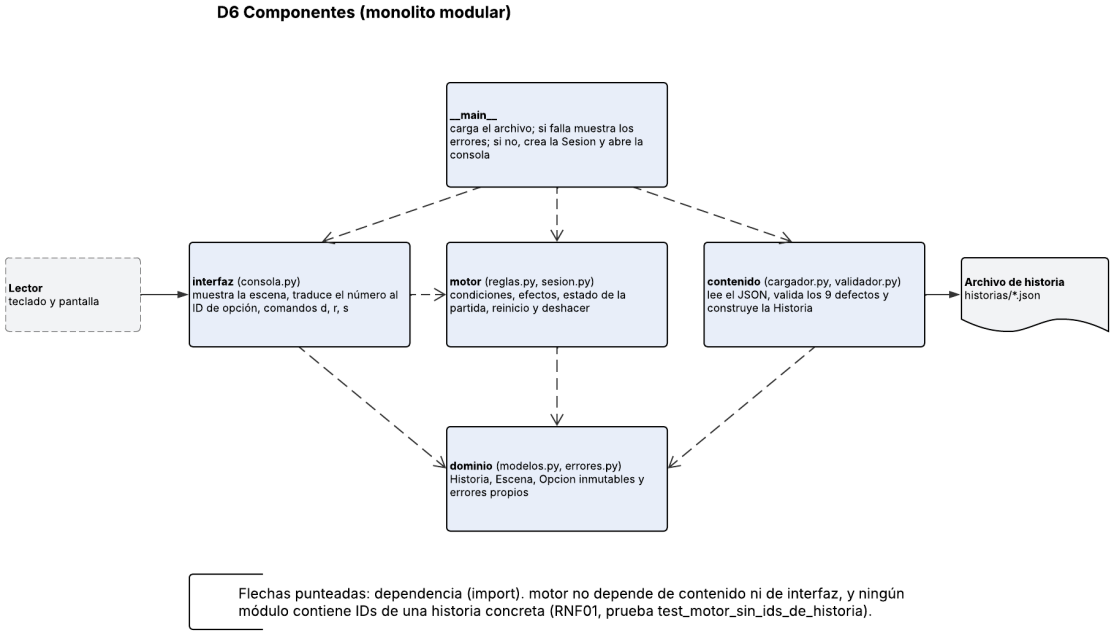


D4. Secuencia: elegir una opción

D5 Estados de una sesión



D5. Estados de una sesión



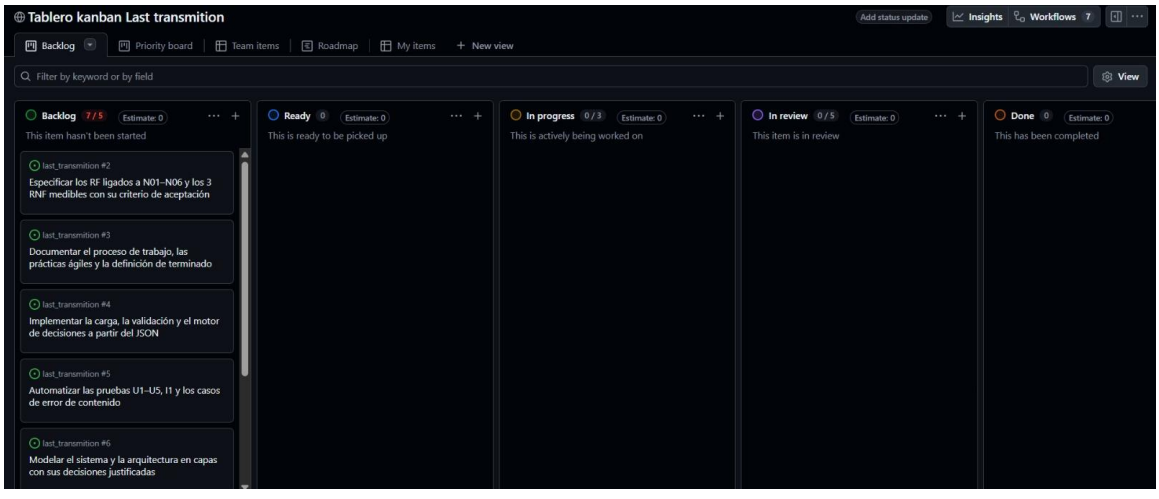
D6. Componentes (monolito modular)

A2. Matriz de trazabilidad

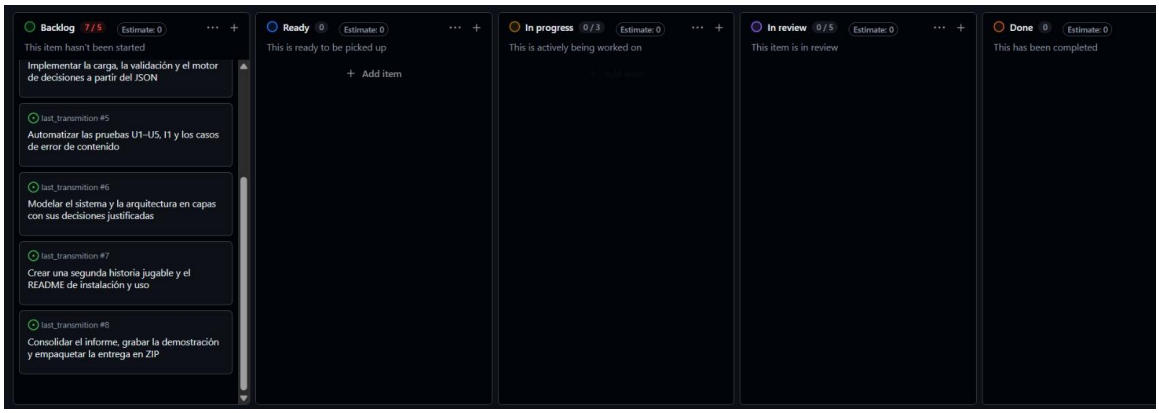
Requisito	Modelos	Componente o archivo (src/transmision/)	Evidencia (tests/)
RF01 Cargar historia	D1, D2, D6	contenido/cargador.py, __main__.py	test_rnf01_segunda_historia_carga_con_el_mismo_motor; test_i1_segunda_historia_funciona_sin_tocar_el_motor
RF02 Inicio y variables	D3, D5	motor/sesion.py (Sesion.__init__)	test_u5_reiniciar_restaura_escena_y_variables
RF03 Texto de la escena	D2, D4	interfaz/consola.py (_mostrar)	test_i1_recorre_los_tres_finales_y_deshace_desde_un_final
RF04 Opciones habilitadas	D3, D4	motor/reglas.py (esta_habilitada), Sesion.opciones_habilitadas	test_u1_condiciones_habilitan_solo_las_correspondientes; test_u1_opcion_sin_condiciones_siempre_se_muestra
RF05 Efectos y destino	D4	motor/reglas.py (aplicar_efectos), Sesion.elegir	test_u2_eleccion_valida_aplica_efectos_y_avanza
RF06 Rechazar opción inválida	D4, D5	Sesion.elegir, dominio/errores.py (OpcionInvalida)	test_u3_opcion_invalida_no_modifica_el_estado; test_rnf02_id_invalido_en_el_motor_es_error_controlado
RF07 Entrada del lector inválida	D4	interfaz/consola.py (ejecutar)	test_rnf02_entrada_de_lector_invalida_no_altera_la_partida
RF08 Conservar variables	D3, D5	Sesion._variables	test_u2_eleccion_valida_aplica_efectos_y_avanza; test_u1_*
RF09 Escena final	D5	Sesion.es_final, consola._mostrar	test_i1_recorre_los_tres_finales_y_deshace_desde_un_final

RF10 Tres finales	D5	historias/la_ultima_transmision.json, Sesion	test_u2_* (E4, E5, E6); test_i1_recorre_los_tres_finales_y_deshace_desde_un_final
RF11 Bloqueo	D5	Sesion.bloqueada, consola._mostrar	test_bloqueo_se_informa_y_permite_reiniciar
RF12 Reiniciar	D2, D5	Sesion.reiniciar	test_u5_reiniciar_restaura_escena_y_variables; test_u5_reiniciar_elimina_el_historial
RF13 Rechazar 9 defectos	D1, D2, D6	contenido/validador.py, contenido/cargador.py	test_u4_archivo_defectuoso_se_rechaza_con_causa_e_id; test_rnf02_archivo_defectuoso_termina_controlado
RF14 Causa y campo o ID	D6	contenido/validador.py, dominio/errores.py (ErrorContenido)	test_u4_archivo_defectuoso_se_rechaza_con_causa_e_id; test_rnf03_mensajes_claros
RF15 Sin partida si se rechaza	D1, D5, D6	__main__.py (main), contenido/cargador.py	test_i1_programa_completo_rechaza_destino_inexistente_sin_traceback
RF20 Deshacer	D2, D3, D4, D5	Sesion.deshacer, Sesion.puede_deshacer, Sesion.elegir, consola (comando d)	test_u6_* (5); test_u5_al_comenzar_no_hay_retroceso; test_u5_reiniciar_elimina_el_historial; test_i1_recorre_los_tres_finales_y_deshace_desde_un_final
RNF01 Modificabilidad	D6	Separación contenido/ motor/ interfaz/; motor sin IDs de escena	test_motor_sin_ids_de_historia; test_rnf01_*; evidencias/rnf01.txt
RNF02 Confiabilidad	D5, D6	__main__.py, contenido/cargador.py, consola.ejecutar	test_rnf02_*; evidencias/pytest_ultima_ejecucion.txt
RNF03 Claridad de errores	D6	contenido/validador.py	test_rnf03_mensajes_claros; evidencias/rnf03_revision.md; docs/mensajes_error.md

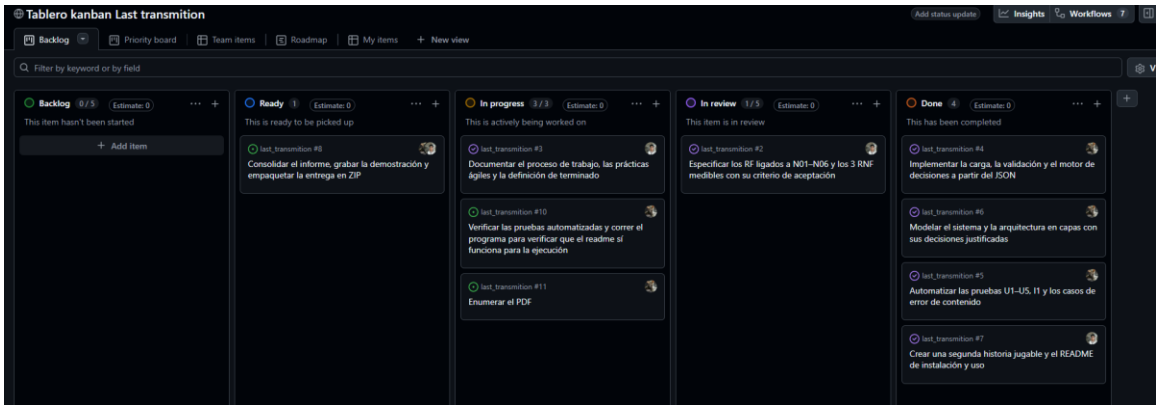
A3. Capturas del tablero



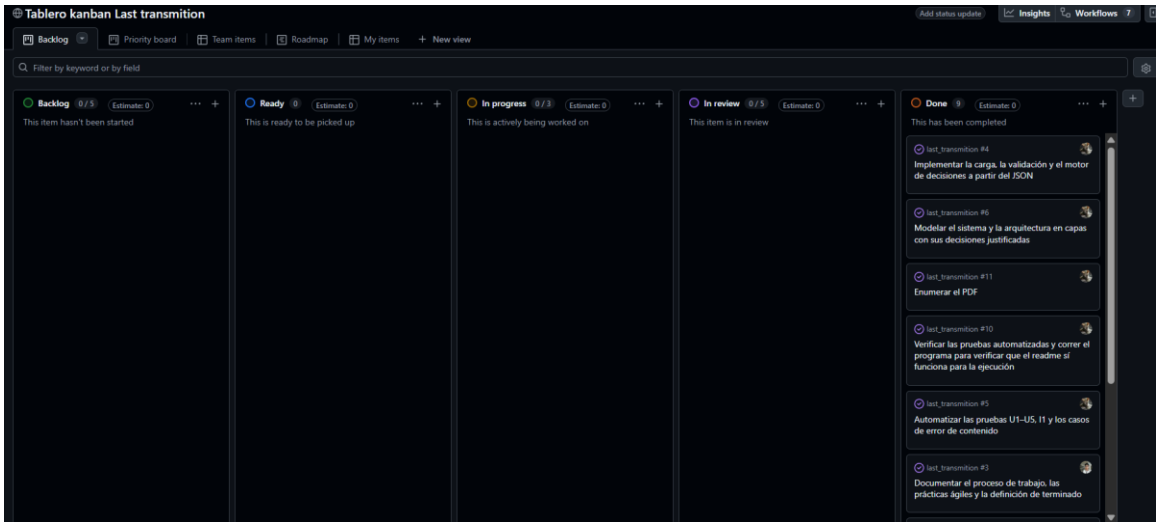
Tablero al inicio (1 de 2)



Tablero al inicio (2 de 2)



Tablero durante la segunda fase (15-10-2026, 11:58)



Tablero final: 9 tarjetas en Done